

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



АВТОМАТИЗОВАНЕ
ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ

Лабораторна робота №5
“Впровадження автоматизованих тестів”

Виконав:
студент гр. КІ-404
Піскурський
С. В.

Прийняв:
Федак П. Р.

Львів – 2024

Завдання:

1. Implement or use existing test framework
2. Create a set of automated tests;
3. Test report should contain number of all tests, passed tests, failed tests, coverage;
4. Coverage must be more than 80%
5. Required steps

Теоретичний матеріал

1. Тестування програмного забезпечення

Тестування — це процес перевірки програмного забезпечення з метою оцінки його якості, виявлення дефектів та забезпечення відповідності системи вимогам.

Основні цілі тестування:

- Виявлення помилок у функціональності чи логіці.
- Підтвердження відповідності програмного забезпечення специфікаціям.
- Підвищення якості програмного забезпечення.

Тестування зазвичай поділяється на кілька рівнів:

- **Модульне тестування (Unit Testing):** Тестування окремих модулів програми.
- **Інтеграційне тестування (Integration Testing):** Перевірка взаємодії між модулями.
- **Системне тестування (System Testing):** Тестування системи як єдиного цілого.
- **Приймальне тестування (Acceptance Testing):** Оцінка програмного забезпечення кінцевими користувачами.

2. Фреймворк GTest

Google Test (GTest) — це популярний фреймворк для написання модульних тестів на C++. Він є простим у використанні, забезпечує широкий набір функціональності та підтримує розширення.

Основні можливості GTest:

- **Тестування функцій та класів:** Фреймворк дозволяє легко створювати тести для окремих функцій та методів.
- **Фікстури тестів (Test Fixtures):** Спрощують ініціалізацію тестових даних, які можна використовувати у кількох тестах.

- **Тестові макроси:** GTest надає макроси для написання тестів, наприклад, TEST, ASSERT_EQ, ASSERT_TRUE.
- **Інтеграція з CI/CD:** GTest легко інтегрується з автоматизованими CI/CD системами для безперервного тестування.

Приклад тесту з GTest:

```
#include <gtest/gtest.h>

// Функція, яку ми тестуємо
int Add(int a, int b) {
    return a + b;
}

// Тестування функції Add
TEST(AdditionTest, PositiveNumbers) {
    EXPECT_EQ(Add(3, 2), 5);
}

TEST(AdditionTest, NegativeNumbers) {
    EXPECT_EQ(Add(-3, -2), -5);
}
```

3. Покриття коду (Code Coverage)

Покриття коду — це метрика, яка показує, яка частина коду була виконана під час тестування. Високе покриття свідчить про те, що більшість коду програми була протестована.

Основні типи покриття:

- **Покриття операторів (Statement Coverage):** Перевірка, чи виконувались всі оператори в коді.
- **Покриття гілок (Branch Coverage):** Перевірка виконання всіх можливих гілок умов (if, switch).
- **Покриття шляхів (Path Coverage):** Перевірка всіх можливих шляхів виконання коду.

Інструменти для вимірювання покриття:

- **gcov:** Стандартний інструмент для покриття в C/C++.
- **Icov:** Графічний інтерфейс для роботи з gcov.
- **LLVM Coverage:** Інструмент для вимірювання покриття в екосистемі LLVM.

Комбінація GTest з Icov дозволяє отримувати звіти про покриття коду.

Виконання:

Опис всіх дій, які були зроблені для додавання підтримки тестів у проект

1. Встановлення GTest

```
cd ~
git clone https://github.com/google/googletest.git
cd googletest
cmake .
make
sudo make install
```

2. Інтеграція GTest з CMake клієнтської частини

```
#-----
# Test configuration
find_package(GTest REQUIRED)

if (GTest_FOUND)
    add_custom_target(tests
        COMMENT "Building and running tests"
    )

    add_custom_command(TARGET tests PRE_BUILD
        COMMAND ${CMAKE_COMMAND} -E echo "Building test executable"
        COMMAND ${CMAKE_COMMAND} --build ${CMAKE_BINARY_DIR} --target game_tests
    )

    set(TEST_SOURCES
        test/test_main.cpp
        test/test_clientCore.cpp
        test/test_cfgFile.cpp
        # test/test_serialUtils.cpp TBD
        src/serialUtils.cpp
        src/clientCore.cpp
        src/cfgFile.cpp
    )

    add_executable(game_tests EXCLUDE_FROM_ALL ${TEST_SOURCES})

    target_link_libraries(game_tests
        PRIVATE
        gmock
        GTest::GTest
        GTest::Main
        ${SERIALPORT_LIB}
    )

    target_compile_definitions(game_tests PRIVATE SERIAL_PORT_PATH="\${SERIAL_PORT_PATH}\")

    target_include_directories(game_tests PRIVATE ${CMAKE_SOURCE_DIR}/include)

    if (CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
        target_compile_options(game_tests PRIVATE --coverage)
        target_link_options(game_tests PRIVATE --coverage)
    endif ()

    enable_testing()
    add_test(NAME RunTests COMMAND game_tests)
else ()
    message("GoogleTest needs to be installed to run the tests")
endif ()
```

3. Додавання тестів у код клієнтської частини (e.g. test_clientCore.cpp)

```
#include <gtest/gtest.h>
#include <gmock/gmock.h>
#include <unordered_map>
#include <cstring>
#include <fstream>

#include "clientCore.h"
```

```

#include "serialUtils.h"

struct sp_port *port; //mock
extern const char* CFG_FILE_NAME;

using namespace testing;

TEST(AIMoveTest, WinningMoveSelected) {
    GameState gameState;
    gameState.current_player = 1;
    gameState.player_1 = 'X';
    gameState.player_2 = 'O';

    // Winning move setup
    strcpy(gameState.board, "XX3456789");

    int move = aiMove(&gameState);

    ASSERT_EQ(move, 2); // Winning move at index 2
    ASSERT_EQ(gameState.board[2], 'X'); // 'X' completes the win
}

TEST(AIMoveTest, BlocksOpponentWinningMove) {
    GameState gameState;
    gameState.current_player = 2;
    gameState.player_1 = 'X';
    gameState.player_2 = 'O';

    // Opponent can win
    strcpy(gameState.board, "XX3456789");

    int move = aiMove(&gameState);

    ASSERT_EQ(move, 2); // AI blocks opponent at index 2
    ASSERT_EQ(gameState.board[2], 'O'); // AI ('O') blocks 'X'
}

TEST(HumanMoveTest, ValidMove) {
    GameState gameState;
    gameState.current_player = 1;
    gameState.player_1 = 'X';
    strcpy(gameState.board, "123456789");

    std::istringstream input("1\n"); // Simulate input for position 1
    std::cin.rdbuf(input.rdbuf());

    int move = humanMove(&gameState);

    ASSERT_EQ(move, 0); // First position (0-indexed)
    ASSERT_EQ(gameState.board[0], 'X'); // 'X' marks position
}

TEST(DisplayBoardTest, CorrectOutput) {
    char board[BOARD_SIZE] = {'1', '2', '3', '4', '5', '6', '7', '8', '9'};

    testing::internal::CaptureStdout();
    displayBoard(board);
    std::string output = testing::internal::GetCapturedStdout();

    ASSERT_NE(output.find(" 1 | 2 | 3 "), std::string::npos);
    ASSERT_NE(output.find(" 4 | 5 | 6 "), std::string::npos);
    ASSERT_NE(output.find(" 7 | 8 | 9 "), std::string::npos);
}

```

4. Інтеграція тестів з CMake серверної частини

```

add_custom_target(test
    COMMAND ${ARDUINO_CLI} compile --fqbn ${BOARD} ${TEST_FILE} --build-property "build.extra_flags=-Iinclude"
    COMMAND ${ARDUINO_CLI} upload -p ${SERIAL_PORT_PATH} --fqbn ${BOARD} ${TEST_FILE}
    COMMAND script -q /dev/null -c "timeout 5 arduino-cli monitor -p /dev/ttyUSB0"
    WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
    COMMENT "Running tests..."
)

```

5. Добавляння тестів у код серверної частини (e.g. test.ino)

```
#include "../include/serverCore.h"
#include "../src/serverCore.cpp"

int passedTests = 0;
int failedTests = 0;

void test_deserializeCMD_InformNewGame() {
    String message = "100_$";
    int result = (int)deserializeCMD(message);
    if (result == (int)REQUEST_CMD::InformNewGame) {
        passedTests++;
        Serial.println("[V]Test deserializeCMD_InformNewGame Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializeCMD_InformNewGame Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializeCMD_Unknown() {
    String message = "999_$";
    int result = (int)deserializeCMD(message);
    if (result == (int)REQUEST_CMD::Unknown) {
        passedTests++;
        Serial.println("[V]Test deserializeCMD_Unknown Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializeCMD_Unknown Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializePosition_ValidInput() {
    String message = "100_4$";
    int result = deserializePosition(message);
    if (result == 4) {
        passedTests++;
        Serial.println("[V]Test deserializePosition_ValidInput Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializePosition_ValidInput Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializePosition_InvalidInput() {
    String message = "100_$";
    int result = deserializePosition(message);
    if (result == 0) { // Expect default (or invalid) position
        passedTests++;
        Serial.println("[V]Test deserializePosition_InvalidInput Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializePosition_InvalidInput Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializeState_CorrectParsing() {
    String message = "100_0_X_0_1_XXXXXXXXX$";
    gamestate_t state;
    deserializeState(message, state);

    if (state.mode == GameMode::ManVsAI &&
        state.player_1 == 'X' &&
        state.player_2 == 'O' &&
        state.current_player == 1 &&
        String(state.board) == String("XXXXXXXXX")) {
        passedTests++;
        Serial.println("[V]Test deserializeState_CorrectParsing Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test deserializeState_CorrectParsing Failed");
    }
}
```

```

}

void test_checkWin_WinningBoard() {
    char board[10] = "XXX.....";
    bool result = checkWin(board);
    if (result) {
        passedTests++;
        Serial.println("[V]Test checkWin_WinningBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkWin_WinningBoard Failed");
    }
}

void test_checkWin_NoWinningBoard() {
    char board[10] = "XX000XXOX";
    bool result = checkWin(board);
    if (!result) {
        passedTests++;
        Serial.println("[V]Test checkWin_NoWinningBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkWin_NoWinningBoard Failed");
    }
}

void test_checkDraw_CompleteBoard() {
    gamestate_t state = {GameMode::ManVsMan, 1, "XX000XXOX", 'X', 'O'};
    bool result = checkDraw(state);
    if (result) {
        passedTests++;
        Serial.println("[V]Test checkDraw_CompleteBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkDraw_CompleteBoard Failed");
    }
}

void test_checkDraw_IncompleteBoard() {
    gamestate_t state = {GameMode::ManVsMan, 1, "XOXOXOX..", 'X', 'O'};
    bool result = checkDraw(state);
    if (!result) {
        passedTests++;
        Serial.println("[V]Test checkDraw_IncompleteBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkDraw_IncompleteBoard Failed");
    }
}

void setup() {
    Serial.begin(9600);
    while (!Serial) {
        ;
    }
    Serial.println("Running tests...");
    delay(100);

    test_deserializeCMD_InformNewGame();
    test_deserializeCMD_Unknown();
    test_deserializePosition_ValidInput();
    test_deserializePosition_InvalidInput();
    test_deserializeState_CorrectParsing();
    test_checkWin_WinningBoard();
    test_checkWin_NoWinningBoard();
    test_checkDraw_CompleteBoard();
    test_checkDraw_IncompleteBoard();

    Serial.print("Tests Passed: ");
    Serial.println(passedTests);
    Serial.print("Tests Failed: ");
    Serial.println(failedTests);

    Serial.println("Test complete.");
}

```

```
void loop() {
    // Nothing needed in loop for now
}
```

6. У build.sh додано окрему функцію для запуску тестів:

```
run_client_tests() {
    echo "Running client tests..."
    mkdir -p "$CLIENT_DIR/build"

    cd "$CLIENT_DIR/build" || exit

    cmake ..
    cmake --build . --target tests

    ./game_tests

    echo "Generating coverage report..."
    touch cov_output.logs
    #kinda workaround with suppressing error logs
    lcov --capture --directory . --output-file coverage.info --ignore-errors mismatch > cov_output.logs
    2>&1
    genhtml coverage.info --output-directory coverage > cov_output.logs 2>&1

    echo "Coverage report generated in: $CLIENT_DIR/build/coverage"

    cd - > /dev/null
    echo "Client tests completed."
}
```

- Додано підтримку опції test:

```
test)
    serial_port=${2:-"/dev/ttyUSB0"} # Default to "/dev/ttyUSB0" if not provided
    run_server_tests "$serial_port"
    run_client_tests
;;
```

7. Перевірка результату

```
serhii@serhii-VirtualBox csad2425ki404piskurskyisv15 bash build.sh test
Running server tests...
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/serhii/csad2425ki404piskurskyisv15/serverSide/build
[100%] Running tests...
Sketch uses 6538 bytes (20%) of program storage space. Maximum is 32256 bytes.
Global variables use 1217 bytes (59%) of dynamic memory, leaving 831 bytes for local variables. Maximum
is 2048 bytes.

Used platform Version Path
arduino:avr 1.8.6 /home/serhii/.arduino15/packages/arduino/hardware/avr/1.8.6
New upload port: /dev/ttyUSB0 (serial)
Using generic monitor configuration.
WARNING: Your board may require different settings to work!

Monitor port settings:
baudrate=9600
bits=8
dtr=on
parity=none
rts=on
stop_bits=1

Connecting to /dev/ttyUSB0. Press CTRL-C to exit.
Running tests...

[V]Test deserializeCMD_InformNewRunning tests...
[V]Test deserializeCMD_InformNewGame Passed
[V]Test deserializeCMD_Unknown Passed
```

```

[V]Test deserializePosition_ValidInput Passed
[V]Test deserializePosition_InvalidInput Passed
[V]Test deserializeState_CorrectParsing Passed
[V]Test checkWin_WinningBoard Passed
[V]Test checkWin_NoWinningBoard Passed
[V]Test checkDraw_CompleteBoard Passed
[V]Test checkDraw_IncompleteBoard Passed
Tests Passed: 9
Tests Failed: 0
Test complete.
[100%] Built target test
Server tests completed.
Running client tests...
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/serhii/csad2425ki404piskurskyisv15/clientSide/build
Building test executable
[100%] Built target game_tests
Built target tests
[=====] Running 6 tests from 4 test suites.
[-----] Global test environment set-up.
[-----] 2 tests from AIMoveTest
[ RUN      ] AIMoveTest.WinningMoveSelected
AI (Player 1(X)) chose position 3
[      OK   ] AIMoveTest.WinningMoveSelected (0 ms)
[ RUN      ] AIMoveTest.BlocksOpponentWinningMove
AI (Player 2(O)) chose position 3
[      OK   ] AIMoveTest.BlocksOpponentWinningMove (0 ms)
[-----] 2 tests from AIMoveTest (0 ms total)

[-----] 1 test from HumanMoveTest
[ RUN      ] HumanMoveTest.ValidMove
Player 1(X) enter your move (1-9): [      OK   ] HumanMoveTest.ValidMove (0 ms)
[-----] 1 test from HumanMoveTest (0 ms total)

[-----] 1 test from DisplayBoardTest
[ RUN      ] DisplayBoardTest.CorrectOutput
[      OK   ] DisplayBoardTest.CorrectOutput (0 ms)
[-----] 1 test from DisplayBoardTest (0 ms total)

[-----] 2 tests from ConfigFileTest
[ RUN      ] ConfigFileTest.ParseValidConfigFile
[      OK   ] ConfigFileTest.ParseValidConfigFile (0 ms)
[ RUN      ] ConfigFileTest.WriteConfigFileUpdatesSection
[      OK   ] ConfigFileTest.WriteConfigFileUpdatesSection (0 ms)
[-----] 2 tests from ConfigFileTest (0 ms total)

[-----] Global test environment tear-down
[=====] 6 tests from 4 test suites ran. (0 ms total)
[ PASSED   ] 6 tests.
Generating coverage report...
Coverage report generated in: clientSide/build/coverage
Client tests completed.

```

8. Інформація про покриття коду:

File:///home/her/h/csd2425ki404piskurskyisv15/clientSide/build/coverage/index.html

LCOV - code coverage report

Current view: top level

Test: coverage.info

Test Date: 2026-11-25 17:50:17

Lines: 94.6 %

Functions: 89.3 %

Coverage

Total

1445

527

Hit

934

365

Directory	Line Coverage %				Function Coverage %			
	Rate	Total	Hit		Rate	Total	Hit	
.	81.0 %	58	47		83.3 %	58	36	
libthis	76.1 %	869	661		85.2 %	298	254	
libtest	50.0 %	34	17		66.7 %	12	8	
game/csd2425ki404piskurskyisv15/clientSide/test	50.0 %	302	97		26.5 %	21	6	
game/csd2425ki404piskurskyisv15/clientSide/test	100.0 %	77	77		100.0 %	28	28	
game/csd2425ki404piskurskyisv15/libthis	100.0 %	2	2		100.0 %	1	1	
game/csd2425ki404piskurskyisv15	17.3 %	81	14		18.5 %	92	17	
game/csd2425ki404piskurskyisv15/external	86.4 %	22	19		88.2 %	17	15	

Generated by LCOV version 2.0.0

Generated by: LCOV version 2.0-1

Висновок:

Використання фреймворку GTest у тестуванні програмного забезпечення забезпечує ефективний підхід до виявлення помилок на ранніх етапах розробки, підвищуючи стабільність та якість коду. Він дозволяє створювати автоматизовані модульні тести, які легко інтегруються в процеси CI/CD для

безперервного тестування. Доповнення процесу інструментами для вимірювання покриття коду, такими як Icov, надає глибоке уявлення про те, які частини програми недостатньо перевірені. Це допомагає гарантувати, що ключові функції та сценарії роботи програмного забезпечення ретельно протестовані, мінімізуючи ризики у фінальному продукті. Після ручного тестування розробленого функціоналу, додав зміни, закомітив їх, створив пул реквест та добавив викладача в рев'юери.